SPARSE MECHANICS
WORLD MODELS

A long-horizon research program for transferable dynamics across interactive worlds

Research program: foundational method → cross-game transfer → AAA validation → generality

---

Core thesis
Learn a compact library of reusable state-transition mechanisms that explains how interactive worlds change after player actions. Predict sparse, causally grounded state updates rather than treating every frame as a dense, independent prediction. Test transfer across genres and AAA environments, while separately measuring predictive accuracy, counterfactual consistency, planning value, and compute.

Program version 2 • Prepared: 12 October 2026

1. Executive summary

This proposal extends sparse world modeling from object-centric manipulation to interactive game environments. The proposed research question is whether a model can learn reusable mechanisms of world change—movement, collision, damage, resource consumption, spawning, destruction, timers, and persistent state transitions—and compose them to predict unseen actions and environments. The objective is not simply to compress video or predict the next frame. It is to preserve action-conditioned state consistency over long horizons while using fewer updates and supporting transfer across games.

The research is organized into two tracks. Track A uses controlled, fast, reproducible environments and public datasets to establish causal claims with ground-truth state and repeatable interventions. Track B evaluates whether the same ideas survive in AAA games: Grand Theft Auto V, Grand Theft Auto: Vice City, and Cyberpunk 2077. AAA experiments should begin with a small, repeatable task suite and synchronized recordings, not an attempt to model each entire game at once.

Recommended primary contribution
A sparse mechanism-transition model: predict (i) which state variables/entities change, (ii) which reusable mechanism caused the change, (iii) the magnitude and timing of the update, and (iv) the downstream effects. Train with paired counterfactual trajectories and evaluate both state prediction and action-outcome consistency.

Research program strategy: this is not a deadline-driven prototype. The objective is to establish whether reusable sparse mechanisms can explain and predict how interactive worlds change, across genres and levels of world complexity. Each phase should be allowed to continue until its scientific exit criteria are met; AAA demonstrations should not substitute for controlled evidence.

Program-wide design principles

- General model first: favor a shared mechanism representation and explicit transfer tests over a separate model for every title. Small adapters are permitted, but their data and parameter budgets must be reported.
- Three distinct evaluation capabilities: predictive accuracy, counterfactual consistency, and interactive planning. Do not infer success on one from results on another.
- AAA as scientific testbeds: GTA V is the first instrumentation target; Vice City and Cyberpunk 2077 are later transfer and robustness tests. Treat game versions, mods, nondeterminism, and available telemetry as explicit experimental variables.
- Falsifiability before scale: state hypotheses, baselines, split rules, and success criteria before collecting large AAA datasets. A negative result about transfer is informative and should be reported.
- Long-term ambition: test transfer across platformers, FPS/action games, racing, open-world games, strategy, and simulation—not just across visually similar titles.

2. Research questions and hypotheses

- RQ1 — Mechanism sparsity: Can a model predict a compact state patch after an action while preserving unaffected state components?
- RQ2 — Transfer: Do reusable mechanisms transfer to unseen levels, maps, game titles, and visual styles better than per-game dynamics?
- RQ3 — Counterfactual consistency: If one action or timing detail changes, can the model predict only the consequential differences between futures?

- RQ4 — Long-horizon reliability: Does sparse mechanism modeling reduce drift and impossible state transitions during long rollouts?
- RQ5 — AAA feasibility: Can the method operate when only pixels and controller inputs are available, and improve when telemetry or mod instrumentation is available?

3. Proposed method

3.1 State representation

Represent the current world as a hybrid state s_t = (E_t, R_t, G_t, q_t, z_t). E_t contains entity-level properties (identity, pose, velocity, health, inventory, animation/state); R_t contains relations (near, inside, attached, targeting, colliding, supporting); G_t contains global variables (weather, time, mission/level state); q_t is player/controller state; z_t is a learned latent state for unobserved mechanics. The exact schema is game-dependent. A visual-only setting estimates these variables with perception and uncertainty; a telemetry-enabled setting uses engine state as privileged supervision.

3.2 Sparse transition patch

Given state and action, predict a change mask m_t, a mechanism assignment c_t, and a state delta Δ_t. The next state is obtained by applying only the predicted patch. Do not impose literal sparsity on every variable: timers, physics, and hidden state may evolve even when no visible object moves. Instead, sparsity should be measured over semantically meaningful updates and causal mechanisms.

Illustrative formulation:

(\hat m_t, \hat c_t, \hat Δ_t) = f (s_t, a_t); \hat s_{t+1} = ApplyPatch(s_t, \hat m_t, \hat Δ_t)

The objective can combine state prediction, event/mechanism prediction, sparse update regularization, counterfactual consistency, and decision-oriented losses. The final formulation should be chosen after ablations; the paper should not rely on a weighted sum of generic losses as its only novelty.

3.3 Reusable mechanism library

- Motion and control response: acceleration, steering, jumping, camera movement, animation transitions.
- Interaction: contact, collision, damage, recoil, knockback, attachment/grasp, vehicle entry/exit.
- Resource/state changes: ammunition, health, stamina, inventory, money, cooldowns, timers, mission flags.

- World evolution: spawning, despawning, destruction, doors/objects changing state, traffic/NPC state changes.
- Game-specific mechanisms: quests, wanted/alert level, checkpoints, race rules, combat phases, scripted events.

The model should infer which mechanisms are active in a particular game and how they compose. Shared mechanism embeddings can capture transferable structure; small per-game adapters can capture control mappings and title-specific rules. Report both fully shared and adapter-based results to avoid overstating zero-shot transfer.

3.4 Counterfactual training

Collect paired rollouts from matched initial states. Change one action, action timing, or environment parameter while holding the rest fixed. Train the model to predict (a) common state components that remain unchanged, (b) the sparse difference set, and (c) the downstream event sequence. This provides a more direct test of causal structure than next-frame prediction alone.

4. Experimental plan

Experiment 1 — Controlled mechanism learning

Goal: establish whether sparse mechanisms improve state prediction and transfer in environments where exact state and actions are known.

- Start with Procgen environments spanning platforming, navigation, combat-like interactions, and resource management; use randomized seeds and held-out level distributions.
- Use Atari/ALE as a broad, lightweight set of discrete-action games, selecting games with different event structures rather than reporting only a single title.
- Use Crafter for longer-horizon survival, crafting, resource, and world-state transitions.
- If available, add a small custom 2D/3D sandbox with controllable collisions, objects, timers, and scripted events to generate paired counterfactuals.

Primary measures: state-variable prediction accuracy; event/mechanism F1; update-mask precision/recall; counterfactual difference accuracy; rollout drift; model size and inference cost.

Experiment 2 — Cross-game transfer

Train on multiple games and hold out entire titles. Evaluate zero-shot transfer, few-shot adaptation, and per-game training. Keep action spaces explicit: a shared latent action representation may help, but do not assume that a button or action has the same semantics across games.

- CrossFPS: use its frame-aligned action telemetry as a focused cross-title FPS test. It contains 69K+ clips across seven FPS titles and 10-DoF controller signals; its listed license is CC BY-NC 4.0, so it is for research/non-commercial use under those terms.
- WildWorld: use a manageable subset for AAA-style action-RPG dynamics. It includes RGB, depth, camera pose, skeletons, and per-frame state/action annotations. The repository reports very large storage requirements, so do not assume the full release is practical for a small-compute project.
- Evaluate title-held-out performance, not only random clip splits; ensure clips from the same session or scene do not leak across train and test.

Experiment 3 — GTA V: first AAA testbed

Recommendation: start with GTA V because offline single-player scripting/modding offers a plausible route to synchronized state and action capture. Use only Story Mode/offline testing; do not run instrumentation in GTA Online. Treat mod APIs as version-dependent and validate the telemetry bridge before collecting a large dataset.

Initial GTA V task suite

- Driving dynamics: accelerate, brake, steer, reverse, take turns, collide lightly, recover, and stop at a waypoint.
- Pedestrian/vehicle interaction: approach, avoid, follow, and observe traffic/NPC reactions in controlled locations.
- Camera and movement: walk/run, turn camera, enter/exit a vehicle, jump, and navigate around obstacles.
- Persistent state: compare state before/after vehicle damage, entering/exiting vehicles, changing wanted/alert state, and moving between locations—only where these transitions can be reliably observed.

Data collection design

- Capture gameplay frames at a fixed target rate and timestamp every controller action. Record resolution, FPS, game build, mod versions, graphics settings, and task script.

- Where legally and technically feasible, instrument player pose/velocity, vehicle pose/velocity, vehicle type, health/damage, wanted level, nearby entities, and mission/interaction state.
- Use deterministic task scripts or carefully reset initial conditions. Collect repeated trials for each action sequence and vary one factor at a time (speed, steering, weather/time, vehicle class, traffic density).
- Maintain two dataset views: (A) visual-only input with pixels/actions; (B) privileged-state input with telemetry. Use B for supervision and diagnostics, but report A as the realistic deployment setting.

Experiment 4 — GTA: Vice City

Use Vice City as a legacy-game and visual/domain-shift test, not as the first environment for developing the method. A community Plugin-SDK supports plugin development for Vice City and related GTA titles, but the actual state fields, compatibility, and collection stability must be verified for the specific game edition.

- Begin with video + timestamped keyboard/controller inputs; add a telemetry plugin only after confirming compatibility and reproducibility.
- Use a compact set of comparable tasks: driving, turning, collisions, pedestrian avoidance, entering/exiting vehicles, and simple navigation.
- Test whether mechanisms learned in GTA V transfer after action remapping and a small visual encoder adaptation. Separate visual appearance transfer from dynamics transfer.
- Do not claim generalization from one game to another if the task semantics or physics differ; report where adaptation is required.

Experiment 5 — Cyberpunk 2077

Treat Cyberpunk 2077 as the most visually complex and instrumentation-sensitive evaluation. REDmod is an official modding toolset, while community tools such as Cyber Engine Tweaks and RED4ext expose additional scripting possibilities; availability and access to specific runtime state depend on game version and tooling. A stable, high-speed, step-able simulator should not be assumed.

Initial Cyberpunk task suite

- On-foot movement and camera control in a fixed area.
- Vehicle driving, turning, braking, collision avoidance, and recovery where the chosen build permits reliable repeatability.

- Combat-lite or target interaction only after safe, reproducible state labels can be collected; record health/ammo/weapon/animation state if accessible.
- World persistence and occlusion: revisit a location, track entities entering/leaving view, and test whether the model distinguishes 'unchanged' from 'unobserved'.

Capture synchronized screen video, input events, timestamps, and any accessible script telemetry. Prefer offline replayable routes and short episodes. For visual-only experiments, infer actions from the controller logger rather than attempting inverse-action inference when inputs can be recorded directly.

5. Dataset plan and access notes

Dataset / environment	**Best use**	**Signals**	**Priority / caveat**
Procgen Benchmark	Fast controlled generalization across procedurally generated game-like tasks	RGB, action, reward, environment state via API	P0; lightweight and reproducible
Atari Learning Environment (ALE)	Broad discrete-action game dynamics and event diversity	RGB, action, reward, emulator state where permitted	P0; game ROM rights/access must be respected
Crafter	Long-horizon survival, crafting, resources, world-state changes	RGB, action, achievement/reward and environment state	P1; useful bridge toward open-world semantics
CrossFPS / SCOPE	Cross-title FPS action-conditioned prediction	Video, frame-aligned 10-DoF actions, captions	P1; listed CC BY-NC 4.0 terms
WildWorld	AAA action-RPG state-aware dynamics and long-horizon evaluation	RGB, depth, camera pose, skeleton, entity/state/action annotations	P1; very large dataset; start with a small permitted subset

Dataset / environment	Best use	Signals	Priority / caveat
GTA V (self-collected)	AAA open-world driving, movement, interaction, persistent state	Video + controller inputs; optional mod telemetry	P1; single-player/offline; build/version-specific
GTA: Vice City (self-collected)	Legacy visual and dynamics domain shift	Video + inputs; optional plugin telemetry	P2; verify edition and plugin support
Cyberpunk 2077 (self-collected)	Photorealistic AAA urban world, driving, movement, occlusion	Video + inputs; optional REDmod/CET telemetry	P2; instrumentation may be fragile; real-time collection is not a fast simulator

6. Baselines and ablations

Required baselines

- Persistence / no-op predictor and simple linear or autoregressive dynamics where applicable.
- Dense latent recurrent dynamics model (e.g., RSSM/Dreamer-style baseline) using the same observations and actions.
- Sparse residual/object-centric baseline from the existing manipulation paper, transferred with minimal changes.
- A generic transformer or video-prediction baseline, where compute permits.
- Per-game model versus shared cross-game model; zero-shot versus few-shot adapter.
- Random-data augmentation versus counterfactual pair mining.

Required ablations

- Remove mechanism library; keep only sparse state patch prediction.
- Remove sparse update mask; use dense state deltas.
- Remove counterfactual training.
- Remove privileged telemetry supervision and train from pixels/actions only.

- Vary sparsity regularization and mechanism granularity.
- Test whether the model benefits from explicit entities/relations versus latent-only state.
- Evaluate short versus long rollouts and seen versus unseen game titles.

7. Metrics

Metric family	**Examples**	**Why it matters**
State prediction	Position/velocity error; discrete state accuracy; entity identity switches	Measures whether the world state is predicted correctly
Sparse updates	Mask precision/recall; update sparsity; false unchanged rate	Tests whether the model changes the right components
Mechanism/event prediction	Event F1; mechanism classification; event timing error	Tests explanatory structure, not just next-frame fit
Counterfactual consistency	Changed-state precision/recall; downstream event agreement	Tests whether action changes produce the correct differences
Long-horizon reliability	Error vs horizon; impossible-transition rate; state drift	Measures compounding error
Transfer	Zero-shot/few-shot performance on held-out titles, levels, and scenarios	Tests the cross-game thesis
Planning/control	Task success; regret vs simulator/oracle; recovery rate	Tests practical value beyond prediction
Efficiency	FLOPs, latency, memory, tokens/state updates per step	Tests whether sparsity creates real efficiency gains

8. AAA data-capture protocol

- Use owned/licensed copies and offline/single-player modes. Do not collect from online multiplayer or bypass anti-cheat systems.
- Create short task scripts with fixed start points, controlled camera orientation, and defined success/failure criteria.

- Synchronize video, controller events, and telemetry with monotonic timestamps; measure and report alignment error.
- Record game version, platform, mods/plugins, settings, frame rate, input device, task seed/reset method, and any known nondeterminism.
- Collect multiple repeats per scenario. Split by session, location, route, and scenario—not random adjacent frames—to avoid leakage.
- Keep raw assets local. Check game EULAs, mod licenses, and dataset licenses before redistributing footage, extracted assets, or telemetry. Prefer publishing code, schemas, annotations derived from permitted sources, and evaluation protocols.

9. Research execution strategy and compute

Do not start by training a foundation-scale video generator. First establish the mechanism-learning claim with compact structured-state and latent-dynamics models. Use the current sparse world-model implementation as the initial baseline, then add the mechanism head and counterfactual data pipeline. Train on low-cost environments first; use a small subset of large datasets and short AAA episodes for final evaluation.

1. Phase A: build data schemas, timestamp alignment, reproducible task scripts, and baseline metrics.
2. Phase B: implement sparse patch + mechanism prediction; verify on Procgen/ALE/Crafter.
3. Phase C: add paired counterfactual collection and training.
4. Phase D: run cross-game held-out evaluations with CrossFPS and/or WildWorld subsets.
5. Phase E: collect GTA V pilot data; expand to Vice City and Cyberpunk after the pipeline is stable.

10. Risks and mitigation

Risk	**Mitigation**
Novelty overlaps with game world models and cross-game action-conditioned video models.	Position the contribution around sparse mechanism transitions + counterfactual state differences + measured transfer; maintain a current related-work matrix.

Risk	Mitigation
Telemetry is unavailable or breaks after updates.	Use video + recorded controller inputs as the mandatory track; treat engine/mod telemetry as optional privileged supervision.
AAA runs are slow and nondeterministic.	Use short routes, repeated trials, fixed settings, matched start states, and offline collection; do not use AAA as the sole scientific testbed.
State extraction errors confound model errors.	Validate detectors/telemetry independently and report both oracle-state and estimated-state results.
Dataset scale exceeds compute/storage.	Use a stratified subset and disclose the sampling protocol; avoid downloading full multi-terabyte datasets unless needed.
Sparse model copies state that should evolve invisibly.	Define semantic update groups and model timers/latent variables explicitly; penalize false unchanged predictions.
Game IP or license restrictions limit release.	Do not redistribute proprietary game assets or circumvention tools; release code, schemas, task definitions, and permitted annotations instead.

11. Research phases and scientific exit criteria

Milestone	Deliverable	Exit criterion
Phase I — Core science	Reproducible controlled benchmark suite; sparse mechanism model; dense/no-op baselines	Improves state/event prediction and update-mask quality at matched data and compute
Phase II — Counterfactuals	Paired trajectories and intervention-based evaluation	Predicts consequential state differences better than non-counterfactual training
Phase III — Cross-game transfer	Multi-title training; held-out-title evaluation; zero-shot and few-shot settings	Demonstrates transfer beyond per-game and shared-dense baselines

Milestone	Deliverable	Exit criterion
Phase IV — AAA pilot	GTA V capture pipeline with synchronized video/actions and optional telemetry	Stable, repeatable tasks; measured timestamp alignment; documented state-label quality
Phase V — External AAA validation	Vice City and Cyberpunk 2077 task suites	Reports transfer/adaptation honestly, including failures and tool/version limitations
Phase VI — Generality	Broader genre suite: platformer, FPS/action, racing, open world, strategy/simulation	Shows which mechanisms transfer, which do not, and the adaptation cost

12. Paper framing, falsifiable claims, and publication strategy

Potential framing: "Sparse Mechanics World Models: Learning Compositional State Transitions Across Interactive Games." Treat this as a research program rather than a deadline-driven prototype. The first paper should establish one central, falsifiable claim: a shared library of sparse state-transition mechanisms improves counterfactual prediction and transfer to held-out games at matched data/compute. Follow-up work can extend the model to richer AAA worlds and more genres. Avoid claiming a universal world model from a handful of titles, or causal understanding solely because a sparse update mask is learned.

Recommended staged scope: establish the method on controlled benchmarks; validate cross-title transfer on a dataset such as CrossFPS and/or a carefully selected WildWorld subset; then use GTA V as the primary AAA pilot. Add Vice City and Cyberpunk 2077 as subsequent external validation once collection is reproducible. This is a sequencing recommendation, not a deadline constraint: each phase should be completed when the evidence is strong enough to justify the next.

12.1 Program-level success criteria

- Scientific: state-change and mechanism predictions outperform strong dense and persistence baselines on held-out conditions.
- Causal/counterfactual: paired interventions yield better predictions of which state components diverge and how effects propagate.
- Transfer: a shared mechanism model beats per-game or generic shared models on held-out titles, with adaptation budget explicitly reported.

- Practical: gains survive closed-loop evaluation and long horizons, not only one-step metrics.
- Efficiency: measure actual latency, memory, FLOPs, and state-update cost; sparsity alone is not evidence of efficiency.
- AAA validity: report task repeatability, telemetry quality, version/mod details, and visual-only versus privileged-state results separately.

13. Datasets, tooling, and references

- **WildWorld dataset and project page:** https://alaya-studio.github.io/wildworld-project/
- **WildWorld dataset repository:** https://github.com/AlayaLab/WildWorld
- **WildWorld on Hugging Face:** https://huggingface.co/datasets/AlayaLab/WildWorld
- **SCOPE paper: cross-game FPS world models:** https://arxiv.org/abs/2605.23345
- **CrossFPS dataset:** https://huggingface.co/datasets/zizhaotong/CrossFPS-train
- **OpenAI Procgen benchmark:** https://github.com/openai/procgen
- **Procgen benchmark announcement:** https://openai.com/index/procgen-benchmark/
- **Arcade Learning Environment:** https://ale.farama.org/
- **Crafter benchmark repository:** https://github.com/danijar/crafter
- **GTA V ScriptHookV community repository (offline modding):** https://github.com/Give-Two/ScriptHookV
- **GTA Vice City Plugin-SDK:** https://github.com/DK22Pac/plugin-sdk
- **Cyberpunk 2077 official modding support / REDmod:** https://www.cyberpunk.net/en/modding-support
- **Cyberpunk modding scripting overview:** https://wiki.redmodding.org/scripting-cyberpunk

Access and licensing note: verify each repository's current license, dataset terms, and game EULA before use or release. The modding links indicate possible instrumentation routes, not a guarantee that every desired state variable is exposed or that a given game build will remain compatible.

Appendix A — Minimum viable experiment matrix

Setting	Train	Test	Primary comparison	Key result
Procgen	Several game types + seeds	Held-out seeds/levels	Sparse vs dense vs no-op	Generalization and update-mask quality
ALE	Subset of games	Held-out games	Shared mechanisms vs per-game models	Cross-game transfer
CrossFPS	6 titles	1 held-out title	Shared model vs title-specific model	Action-conditioned transfer
WildWorld subset	Selected scenes/actions	Held-out scene/species/weapon where split permits	State-aware dense vs sparse mechanics	Long-horizon state consistency
GTA V	Driving/movement tasks	New routes, vehicles, weather/time conditions	Video-only vs telemetry-supervised	AAA dynamics prediction
Vice City	Small pilot / optional adapter	New routes or action variants	Zero-shot vs few-shot adaptation	Legacy-domain transfer
Cyberpunk 2077	Fixed routes + movement tasks	New routes/occlusions/conditions	Visual-only vs optional telemetry	Photorealistic external validation